

Welcome To Complete SQL Course

Beginner to Advanced



MySQL Topics Covered In This Video:

1. Introduction of MySQL | Database | RDBMS
2. Installation of MySQL
3. MySQL Commands | Data Types | Database Query
4. Naming Conventions | Comments | CREATE & INSERT Command
5. SQL Constraints
6. ALTER & SELECT Command
7. WHERE Clause | UPDATE & DELETE Command | Auto Increment | LIMIT & OFFSET
8. ORDER By | LIKE Operations
9. OPERATORS IN SQL - Arithmetic, Comparison & Logical
10. Functions in SQL
11. Group By & Having | Group_Concat
12. Subquery
13. Case Statement
14. Joins
15. CROSS Join & SELF Join
16. Window Functions in SQL
17. UNION Vs UNION ALL in SQL
18. Common Table Expression(CTE)
19. Recursive Common Table Expression(CTE)
20. Views(Virtual Table)
21. Indexes
22. Pivot and Unpivot

Video-1

Introduction of MySQL

Topics to Cover:

- What is a Database?
- What is DBMS?
- What is RDBMS?
- What is MySQL?
- SQL Vs MySQL
- MySQL Workbench
- Why I choose MySQL?



What is a Database ?

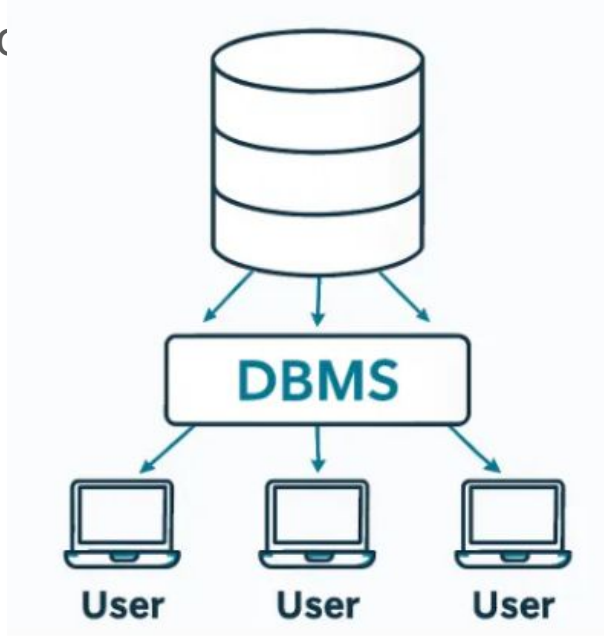
A database is a structured collection of data that allows efficient storage and retrieval of information.

🎓 Example of database : Student data

 StudentID	 Name	 Age	 Grade	 Email	 City
1001	Aditi Sharma	16	10th	aditi.sharma@email.com	New Delhi
1002	Rohan Mehta	17	11th	rohan.mehta@email.com	Mumbai
1003	Sneha Verma	15	9th	sneha.verma@email.com	Bengaluru
1004	Arjun Kapoor	18	12th	arjun.kapoor@email.com	Hyderabad
1005	Priya Nair	16	10th	priya.nair@email.com	Chennai

DBMS

Database is managed by a Database Management System (DBMS), which processes data requests, and manage data efficiently.



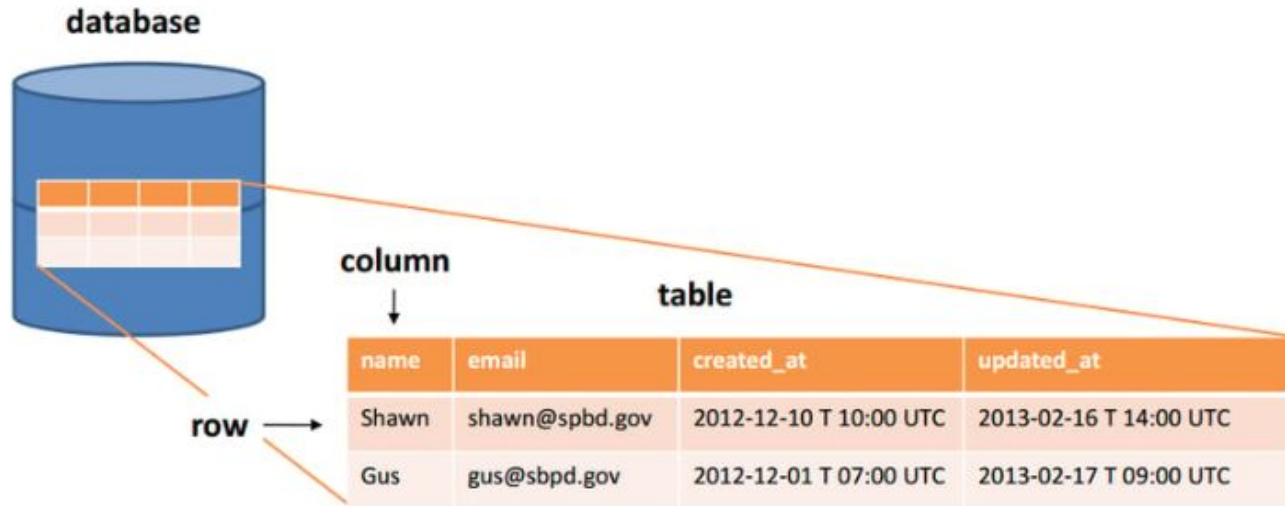
What is RDBMS ?

RDBMS stands for Relational Database Management System — a type of DBMS where data is stored in tables.

- Each table consists of rows (records) and columns (fields).

- Tables are related to each other by relationships.

relationships



Example of DBMS:



What is SQL?

SQL is Structured Query Language - used to store, manipulate and retrieve data from RDBMS.

It is used to perform CRUD operations:

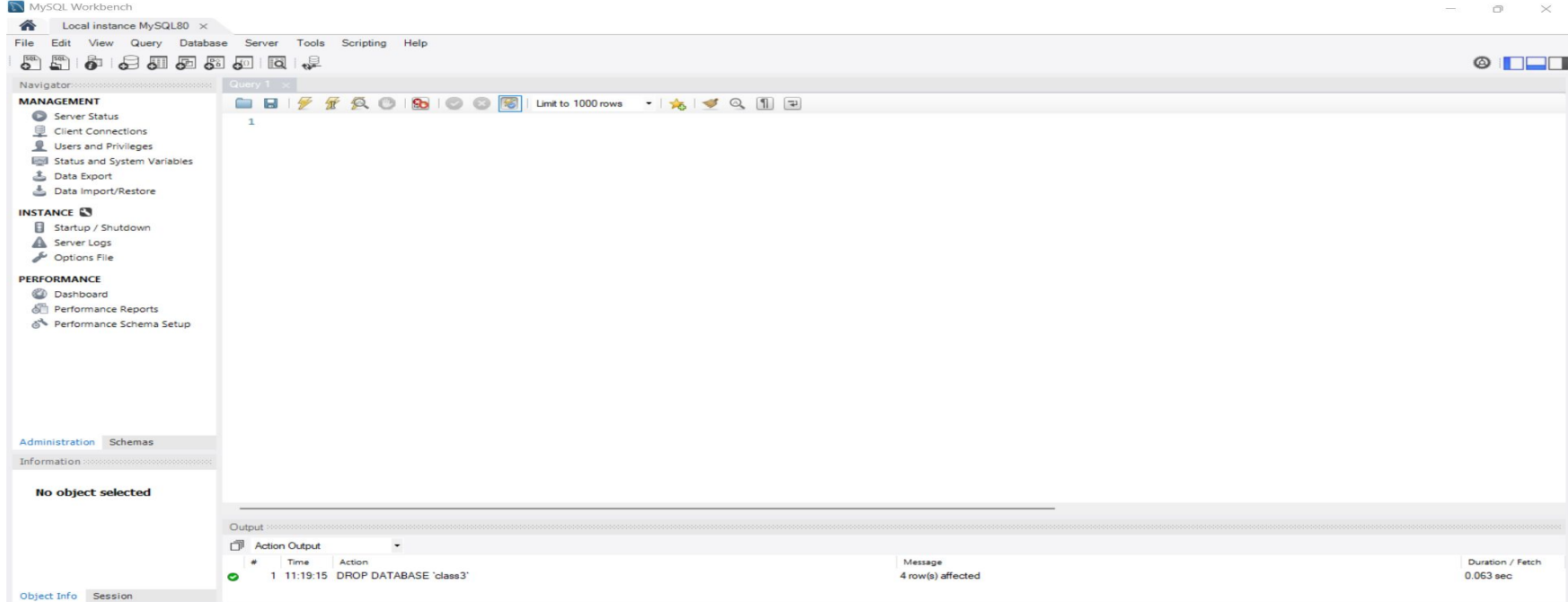
- Create
- Read
- Update
- Delete

SQL Vs MySQL ?

SQL is Structured Query Language - It is not a database, it is a language used to interact with database.

MySQL is a RDBMS which uses SQL to talk with database.

MySQL Workbench?



Why I choose MySQL ?

which database is popular to learn in 2025

AI Mode

All

Images

Short videos

Videos

News

Forums

More ▾

Tools ▾

✦ AI Overview

हि

En



Listen

Popular databases to learn in 2025 include **MySQL** and **PostgreSQL** for general-purpose, relational database skills, **MongoDB** for NoSQL applications, and **vector databases** like **Pinecone** for AI and machine learning. Choosing a database depends on your specific career goals, as many popular options exist for different use cases, from traditional web applications to modern, data-intensive AI projects. [🔗](#)

Video-2

Installation of MySQL

Video Link : https://youtu.be/Dvy-hR16APc?si=_x0ZpuwQuwXpAVv5

Video-3

Topics to Cover:

- Types of MySQL Commands
- Data Types in MySQL
- MySQL Keyword Case Sensitivity
- Why Semicolons Matter in MySQL
- Query related to Database

Types of SQL Commands: (Important Interview Question)

1. DDL(Data Definition Language) - it consists of SQL commands that can be used to define the database structures but not data.

- Create - create database, table, index, views, store procedure and triggers
- Drop - drop database, table etc
- Alter - alter the structure of database like add/drop/rename column, modify datatype.
- Truncate - remove all the records from the table.
- Rename - rename database, table etc

2. DML(Data Manipulation Language) -The SQL commands that deal with the manipulation of data present in the database belong to DML and this includes most of the SQL statements.

- **INSERT:** It is used to insert data into a table.
- **UPDATE:** It is used to update existing data within a table.
- **DELETE:** It is used to delete records from a database table.

3. DQL(Data Query Language) - It includes select commands.

4. DCL(Data Control Language) - It includes commands such as GRANT and REVOKE.

which mainly deal with the rights, permissions, and other controls of the database system.

- **GRANT** : This command gives users access privileges to the database.
- **REVOKE** : This command withdraws the user's access privileges given by using the GRANT command.

Data Types in MySQL:

Sure! Here's a brief overview of the main MySQL data types, grouped by category for quick reference:

1. **Numeric Types** : Used for storing numbers.

- INT – Whole numbers (e.g., 1, 1000)
- TINYINT, SMALLINT, MEDIUMINT, BIGINT – Smaller or larger ranges of integers
- DECIMAL – Exact numeric values (great for money)
- FLOAT, DOUBLE – Approximate numeric values (used in scientific calculations)

2. String Types : Used for storing text.

- CHAR(n) – Fixed-length string
- VARCHAR(n) – Variable-length string
- TEXT – Large text data
- TINYTEXT, MEDIUMTEXT, LONGTEXT – Increasing sizes of text fields

3. Date & Time Types : Used for storing dates and times.

- DATE – Only date (YYYY-MM-DD)
- DATETIME – Date and time
- TIMESTAMP – Auto-updating time stamp
- TIME – Time only
- YEAR – Year value

SQL Keyword Case Sensitivity

SQL keywords are NOT case sensitive

- select is the same as SELECT

Why Semicolons Matter in MySQL

- Some database systems require a semicolon at the end of each SQL statement.
- Semicolon is the standard way to separate each SQL statement.

Query related to Database:

Create Database :

Syntax: Create Database Database_Name;

Example: Create Database first_db;

To see all the existing Database

Show Databases;

Go inside the database

Syntax: use Database_Name;

Example: Use first_db;

Deleting a database

Syntax: Drop Database Database_Name;

Example: Drop Database first_db;

Video-4

Topics to Cover:

- Recommended Naming Conventions for MySQL Database/Table
- Comments in MySQL
- Create Table in MySQL
- Insert Data in Table

Recommended Naming Conventions for MySQL Database/Table

1. Use lowercase letters:

Example: sales_db, not Sales_DB

2. Use underscores to separate words: Enhances readability.

Example: customer_data, inventory_system

3. Avoid special characters and spaces: Stick to letters, numbers, and underscores.

❌ Avoid: my database, data@2025

4. Start with a letter:

✅ Good: analytics_db

❌ Bad: 123data

5. Keep names short but descriptive: Avoid overly long or cryptic names.

Example: hr_portal, not hrp or human_resource_management_portal_v1

6. Avoid Reserved Words

Don't use SQL keywords like SELECT, TABLE, or DATABASE as names.

By Amit Kumar

Types of Comments in MySQL

1. Single-line comment using --

-- This is a single-line comment

2. Single-line comment using

Another way to write a single-line comment

3. Multi-line comment using /*.....*/

/* this is a

Multi line

comment*/

Create Table Command

The CREATE TABLE statement is used to create a new table in a database.

Syntax:

```
CREATE TABLE table_name  
(  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
)
```

Example:

```
Create Table employee  
(  
    id int,  
    name varchar(30)  
);
```

Create Table Command

Another way of writing it:

Create Table if not Exists Employee

```
(  
    id int,  
    name varchar(30)  
);
```

Create emp table with few more column

create table employee1

```
(  
    id int,  
    name varchar(30),  
    salary double,  
    hiring_date Date  
);
```

Insert Data Command

The INSERT INTO statement is used to insert new records in a table

Syntax

```
INSERT INTO TABLE_NAME
```

```
(column1, column2, column3,... columnN )
```

```
VALUES
```

```
(value1, value2, value3,... valueN);
```

Example:

```
Insert into employee1
```

```
(id, name, salary, hiring_date)
```

```
Values
```

By Amit Kumar

Insert Data Command

Note : You can omit column names if you're inserting into all columns in order

Example:

Insert into employee1

Values

```
(1,"Amit",1000,"2025-08-01");
```

What if you not follow the order : interchange id column with name column

Insert into employee1

Values

```
("Ankit",1,1000,"2025-08-01");
```

Insert Data Command

What if you skip any column value: skip id column value

Insert into employee1

Values

("Amit",1000,"2025-08-01");

Insert multiple records into a table

Insert into employee1

Values

(2,"ashu",5000,"2021-10-28"),

(3,"nitin",3500,"2021-09-16"),

(4,"kajal",2900,"2021-09-20");

Video-5

Topics to Cover:

- SQL Constraints

SQL Constraints

- SQL constraints are used to specify rules for the data in a table.
- Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

The following constraints are commonly used in SQL:

1. NOT NULL
2. UNIQUE
3. PRIMARY KEY
4. FOREIGN KEY
5. CHECK
6. DEFAULT

Why we need Constraints ?

Example: - table without Constraints

```
Create table employee (  
    id int,  
    name varchar(30),  
    salary double  
);
```

```
DESC employee;
```

insert data without name

```
insert into employee (id, salary)  
Values  
(1,3000);
```

insert data without salary and name

```
insert into employee (id)
```

```
Values
```

```
(1);
```

SQL Constraints

1. **NOT NULL** - Ensures that a column cannot have a NULL value.

Example: - In this example, every student must have a name.

```
Create table employee1 (  
    id int,  
    name varchar(30) not null,  
    salary double  
);
```

```
DESC employee1;
```

Column name can not be null

```
insert into employee1(id, salary)  
Values  
(1,3000);
```

Column name can not be null

```
insert into employee1
```

SQL Constraints

2. UNIQUE - Ensures that all values in a column are different.

Example: - In this example, every student must have a name.

```
Create table employee2 (  
    id int Unique,  
    name varchar(30),  
    salary double );
```

```
Select * from employee2;
```

It will allow this

```
insert into employee2
```

Values

```
(1,"Rajan",3000);
```

ID can not be duplicate

```
insert into employee2
```

Values

SQL Constraints

3. Check- Ensures values meet a specific condition.

Example: - In this example, every student must have a name.

```
Create table employee3 (  
    id int,  
    name varchar(30),  
    salary double check (salary>1000)  
);
```

```
Select * from employee3;
```

It will allow this

```
insert into employee3
```

Values

```
(1,"Sumit",3000);
```

salary can not be less than 1000

```
insert into employee3
```


SQL Constraints

4. Default- Assigns a default value if none is provided.

Example: - In this example, every student must have a name.

```
Create table employee4 (  
    id int,  
    name varchar(30),  
    hiring_date Date default "2023-08-11"  
);
```

```
Select * from employee4;
```

It will allow this

```
insert into employee4 (id, name, hiring_date)
```

Values

```
(1,"Sumit","2022-01-01");
```

it will also allow this and take default value in hiring_date column

```
insert into employee4 (id, name)
```

By Amit Kumar

SQL Constraints

5. **Primary Key** - Uniquely identifies each row in a table.

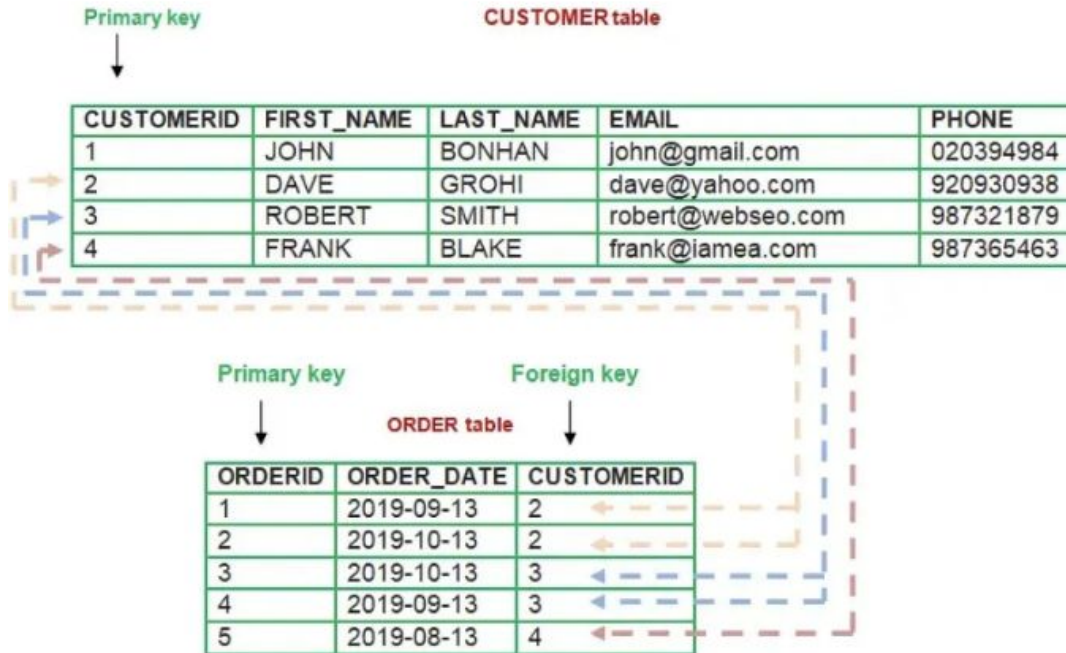
- It is a combination of NOT NULL and UNIQUE.
- A table can have only one primary key.

Example:

```
create table customer (  
    cust_id int PRIMARY KEY,  
    name VARCHAR(50),  
);
```

6. **Foreign Key** - foreign key creates a link between two tables by referencing the primary key of another table.

A primary key-foreign key relationship



Primary Key & Foreign Key

```
create table customer (  
    cust_id int,  
    name VARCHAR(50),  
    age int,  
    constraint pk1 Primary Key (cust_id)  
);  
DESC customer;
```

```
create table orders (  
    order_id int,  
    order_num int,  
    customer_id int,  
    constraint pk2 Primary Key (order_id),  
    constraint fk1 Foreign Key (customer_id) REFERENCES customer(cust_id)  
);  
DESC orders;
```

Insert record in customer table

```
insert into customer values(1,"Akash",29);
```

```
insert into customer values(2,"Rahul",30);
```

```
select * from customer;
```

Insert record in orders table

```
insert into orders values(1001, 20, 1);
```

```
insert into orders values(1002, 30, 2);
```

```
insert into orders values(1003, 40, 2);
```

```
select * from orders;
```

try to insert a record in foreign key table with a id which is not present in primary key table

```
insert into orders values(1004, 35, 3);
```

Two ways to declare the constraints

1. At the time of variable declaration

```
Create table employee_with_constraints (  
    id int UNIQUE,  
    name varchar(30),  
    salary double CHECK (salary >1000)  
);
```

2. In the last line

```
Create table employee_with_constraints1 (  
    id int,  
    name varchar(30),  
    salary double,  
    CONSTRAINT unique_emp_id UNIQUE (id),  
    CONSTRAINT salary_check CHECK (salary > 1000)  
);
```

Note: we can omit the constraint keyword and its name, but we have to provide full naming to our constraints so it is easy to

debug in case of error

Video-6

Topics to Cover:

- ALTER Command
- SELECT Command

ALTER Command

The ALTER command in SQL is used to modify the structure of an existing table—such as adding, deleting, or changing columns and constraints.

What ALTER TABLE Can Do:

The ALTER TABLE statement belongs to the Data Definition Language (DDL) and allows you to:

- Add new columns
- Delete existing columns
- Modify column data types
- Rename columns or the table
- Add or drop constraints (e.g., primary key, foreign key, unique)

ALTER Commands

```
CREATE DATABASE class_db;
```

```
USE class_db;
```

```
CREATE TABLE employee (  
    Id int,  
    name varchar(50),  
    age int  
);
```

```
INSERT INTO employee  
Values (1, "Mohit", 31);
```

```
SELECT * FROM employee;
```

Add new column named DOB in the table

```
ALTER TABLE employee  
ADD COLUMN dob date;
```

Note: Column keyword is optional we can omit that also

```
DESC employee;  
select * from employee;
```

Modify existing column in a TABLE or change datatype of a column or increase length of a column

```
ALTER TABLE employee  
MODIFY name varchar(100);
```

Delete existing column from given TABLE or remove city column from employee table

```
ALTER TABLE employee  
DROP COLUMN dob;
```

Rename the column name to full_name

```
ALTER TABLE employee
```

```
RENAME COLUMN name to full_name;
```

```
select * from employee;
```

Difference between Drop & Truncate Command

```
INSERT INTO employee values(2,"Rahul", 25);
```

```
INSERT INTO employee values(3,"Sunny", 22);
```

Truncate : TRUNCATE deletes only the data inside the table but keeps the structure intact.

```
TRUNCATE TABLE employee;
```

DROP: DROP removes the entire table including its structure.

```
DROP TABLE employee;
```

Alter command on integrity constraint

```
Create table employee1 (  
    Id int,  
    Name varchar(50),  
    Age int,  
    Hiring_date date,  
    Salary int  
);
```

```
insert into employee1 values(1,"Ankit", 24, "2021-08-10", 10000);  
insert into employee1 values(2,"Rahul", 25, "2021-08-10", 20000);  
insert into employee1 values(3,"Sunny", 22, "2021-08-11", 11000);  
insert into employee1 values(4,"Amit", 25, "2021-08-11", 12000);  
insert into employee1 values(5,"Mohit", 26, "2021-08-12", 50000);
```

```
select * from employee1;
```

Add unique integrity constraint on id COLUMN

```
alter table employee1  
add constraint id_unique unique(id);
```

```
desc employee1;
```

It will not allow us to insert this record bcz of unique integrity constraint

```
insert into employee1
```

Values

```
(1,"Dheeraj", 24, "2021-08-10", 10000);
```

Drop constraint from existing Table

```
alter table employee1
```

```
drop constraint id_unique;
```

SELECT Command

The SELECT command in SQL is used to retrieve data from one or more tables in a database. It's the most commonly used SQL statement for querying data.

Syntax:

Select All Columns:

```
SELECT *
```

```
FROM table_name;
```

Select Specific columns:

```
SELECT column1, column2, ...
```

```
FROM table_name;
```

SELECT Commands

display all columns in the final result

```
select * from employee1;
```

display specific columns in the final result

```
select name, age  
from employee1;
```

Count total records

```
select count(*)  
from employee1;
```

Alias declaration

```
select count(*) as total_row_count  
from employee1;
```

Aliases for multiple columns

```
select name as employee_name, age as employee_age  
from employee1;
```

Print unique hiring_dates from the employee table when employees joined it.

```
select Distinct(hiring_date) as distinct_hiring_dates  
from employee1;
```

How many unique age values are in the table?

```
select count(distinct(age)) as total_unique_ages  
from employee1;
```

Increment salary of each employee by 20% and display final result with new salary

```
select *, (salary*1.2) as new_salary  
from employee1;
```


Video-7

Topics to Cover:

- WHERE Clause
- UPDATE Command
- DELETE Command
- Use of Auto Increment
- Use of LIMIT & OFFSET

Topics to cover:

- WHERE Clause
- UPDATE Command
- DELETE Command
- Use of Auto Increment
- Use of LIMIT & OFFSET

WHERE Clause

The **WHERE** clause in SQL is used to filter records that meet specific conditions. It's commonly used with **SELECT**, **UPDATE**, **DELETE**, and other statements to target only the rows you want.

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Example: SELECT with WHERE

```
SELECT * FROM employee  
WHERE age = 25;
```

Example: SELECT with WHERE

```
SELECT * FROM employee  
WHERE hiring_date = "2021-08-10";
```

Creating employee table to understand different Commands

```
Create table employee (  
    Id int,  
    Name varchar(50),  
    Age int,  
    Hiring_date date,  
    Salary int  
);
```

```
insert into employee values(1,"Ankit", 24, "2021-08-10", 10000);  
insert into employee values(2,"Rahul", 25, "2021-08-10", 20000);  
insert into employee values(3,"Sunny", 22, "2021-08-11", 11000);  
insert into employee values(4,"Amit", 25, "2021-08-11", 12000);  
insert into employee values(5,"Mohit", 26, "2021-08-12", 50000);
```

```
select * from employee;
```

```
desc employee;
```

UPDATE Command

The UPDATE command in SQL is used to modify existing records in a table.

Syntax:

```
UPDATE table_name
```

```
SET column1 = value1, column2 = value2, ...
```

Syntax with where clause:

```
UPDATE table_name
```

```
SET column1 = value1, column2 = value2, ...
```

```
WHERE condition;
```

UPDATE Command

Updates will be made for all rows:

```
UPDATE employee  
SET age = 20;
```

```
SET SQL_SAFE_UPDATES = 0;
```

```
select * from employee;
```

Update the salary of employee after giving 20% increment

```
UPDATE employee  
SET salary = salary * 1.2;
```

Update command for multiple columns

```
UPDATE employee  
SET salary = 1.2, age=24;
```

UPDATE Command

Update a single column

```
UPDATE employee  
SET salary = 30000  
WHERE id = 1;
```

Update multiple columns

```
UPDATE employee  
SET salary = 80000, age = 30  
WHERE id = 3;
```

DELETE Command

The **DELETE** command in SQL is used to remove one or more rows from a table based on a condition.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

Delete a specific record

```
DELETE FROM employee  
WHERE id = 1;
```

Delete multiple records

```
DELETE FROM employee  
WHERE Hiring_date = "2021-08-11";
```

Delete all rows (you can also use truncate here as truncate is much faster than delete)

```
DELETE FROM employee;
```


Auto increment

The **AUTO_INCREMENT** attribute in SQL is used to automatically generate a unique value for a column whenever a new row is inserted. It's commonly applied to primary key columns like **id**.

Syntax:

```
CREATE TABLE employees (  
  id INT AUTO_INCREMENT,  
  name VARCHAR(100),  
  PRIMARY KEY (id)  
);
```

Each time you insert a new row, employee_id will automatically increment from the last value.

```
INSERT INTO employees (name)  
VALUES ('Ankit'), ('Mohit');
```

Auto increment

```
insert into employees (id, name)  
values(5,"Amit");
```

```
insert into employees (name)  
values("Nikhil");
```

```
Select * from employees;
```

Use of LIMIT & OFFSET

The **LIMIT** and **OFFSET** clauses in SQL are used to control how many rows are returned and where to start retrieving them.

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
LIMIT number_of_rows OFFSET start_position;
```

Get the first 3 rows:

```
SELECT * FROM employee  
LIMIT 3;
```

Skip the first 2 rows and return the next 3

```
SELECT * FROM employee  
LIMIT 3 OFFSET 2;
```

Alternative Syntax

```
SELECT * FROM employee  
LIMIT 2, 3;
```

Video-8

Topics to Cover:

- Order By
- Like Operations

Topics to cover:

- Order By
- Like Operations

ORDER BY Clause

The ORDER BY clause in SQL is used to sort the result set of a query by one or more columns, either in ascending (ASC) or descending (DESC) order.

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ...;
```

- ASC (default): Sorts from lowest to highest (A–Z, 0–9).
- DESC: Sorts from highest to lowest (Z–A, 9–0).

Creating employee table to understand different Commands

```
Create table employee (  
    Id int,  
    Name varchar(50),  
    Age int,  
    Hiring_date date,  
    Salary int  
);
```

```
insert into employee values(1,"Ankit", 24, "2021-08-10", 10000);  
insert into employee values(2,"Arya", 25, "2021-08-10", 20000);  
insert into employee values(3,"Sunny", 22, "2021-08-11", 11000);  
insert into employee values(4,"Amit", 25, "2021-08-11", 12000);  
insert into employee values(5,"Mohit", 26, "2021-08-12", 50000);  
insert into employee values(6,"Sumit", 26, "2021-08-12", 50000);
```

```
Select * from employee;
```

Examples

1. Sort by one column (ascending)

```
SELECT * FROM employee  
ORDER BY name;
```

2. Sort by one column (descending)

```
SELECT * FROM employee  
ORDER BY name DESC;
```

3. Sort by multiple columns

```
SELECT * FROM employee  
ORDER BY age DESC, name ASC;
```

This sorts first by age (highest first), then by name (A–Z) within each age;

Examples

4. You can use column aliases in ORDER BY

```
SELECT *, salary * 12 AS annual_salary  
FROM employee  
ORDER BY annual_salary DESC;
```

5. You can use column positions in ORDER BY

```
SELECT name, age FROM employee  
ORDER BY 2 DESC; -- Sorts by the second column (age)
```

LIKE Operations

The **LIKE** operator in SQL is used to search for a specific pattern in a column's text data, often with wildcards like **%** and **_** to match partial strings.

Syntax

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE column_name LIKE pattern;
```

Wildcard	Meaning
%	Matches zero, one, or many characters
_	Matches exactly one character

Examples

1. Get all those employees whose name starts with "A".

```
SELECT * FROM employee  
WHERE name LIKE "A%";
```

2. Get all those employees whose name starts with "SU".

```
SELECT * FROM employee  
WHERE name LIKE "SU%";
```

3. Get all those employees whose name ends with "T".

```
SELECT * FROM employee  
WHERE name LIKE "%T";
```

4. Get all those employees whose name starts with "A" and ends with "T".

```
SELECT * FROM employee  
WHERE name LIKE "A%T";
```

Examples

5. Get all those employees whose name will have exact 5 characters.

```
SELECT * FROM employee  
WHERE name LIKE "_____";
```

6. Return all those employees whose name contains at least 5 characters.

```
SELECT * FROM employee WHERE name LIKE "%_____";  
SELECT * FROM employee WHERE name LIKE "_____%";  
SELECT * FROM employee WHERE name LIKE "%_____%";
```

Video-9

Topics to Cover:

- OPERATORS IN SQL
 - Arithmetic Operators
 - Comparison Operators
 - Logical Operators

1. Arithmetic Operators

Used for basic math operations:

- + : Addition
- - : Subtraction
- * : Multiplication
- / : Division
- % : Modulo (remainder)

Creating employee table to understand different Commands

```
CREATE TABLE employees (  
  id INT,  
  name VARCHAR(50),  
  age INT,  
  salary DECIMAL(10,2),  
  department VARCHAR(50),  
  bonus INT  
);
```

```
INSERT INTO employees VALUES  
(1, 'Amit', 28, 30000.00, 'HR', 3000),  
(2, 'Neha', 20, 17000.00, 'IT', 1700),  
(3, 'Raj', 24, 20000.00, 'Finance', 2000),  
(4, 'Simran', 29, 25000.00, 'IT', 2500);
```

Arithmetic Operators Example:

Calculate total compensation by adding salary to bonus.

```
SELECT name, salary + bonus AS total_compensation  
FROM employees;
```

Deduct 1000 from each employee salary.

```
SELECT name, salary - 1000 AS new_salary  
FROM employees;
```

Multiplication

```
SELECT 10 * 2;
```

Division

```
SELECT 10 / 2;
```

Modulo (remainder)

```
SELECT 10 % 3;
```


2. Comparison Operators

Used to compare values:

- > : Greater than
- < : Less than
- >= : Greater than or equal to
- <= : Less than or equal to
- = : Equal to
- != or <> : Not equal to
- BETWEEN : Within a range

Comparison Operators Example

list all employees who are getting salary more than 20000.

```
SELECT * FROM employees
```

```
WHERE salary > 20000;
```

list all employees who are getting salary more than or equal to 20000.

```
SELECT * FROM employees
```

```
WHERE salary >= 20000;
```

list all employees who are getting less than 20000.

```
SELECT * FROM employees
```

```
WHERE salary < 20000;
```

list all employees who are getting salary less than or equal to 20000.

```
SELECT * FROM employees
```

```
WHERE salary <= 20000;
```

list all employees where age of employees is equal to 20.

```
SELECT * FROM employees
```

```
WHERE age = 20;
```

Comparison Operators Example

list all employees where age of employees is not equal to 20.

```
SELECT * FROM employees  
WHERE age != 20;
```

```
SELECT * FROM employees  
WHERE age <> 20;
```

list all employees who are getting salary in the range of 15000 to 28000.

```
SELECT * FROM employees  
WHERE salary BETWEEN 15000 AND 28000;
```

3. Logical Operators

Used to combine multiple conditions:

- AND : All conditions must be true
- OR : At least one condition must be true
- NOT : Negates a condition

Logical Operators Example

Both conditions must be true.

```
SELECT * FROM employees  
WHERE age > 25 AND department = 'IT';
```

Either condition can be true.

```
SELECT * FROM employees  
WHERE department = 'HR' OR department = 'Finance';
```

Example of NOT (Excludes IT department).

```
SELECT * FROM employees  
WHERE NOT department = 'IT';
```

Topics to cover:

- Functions in SQL

Video-10

Topics to Cover:

- Functions in SQL

Functions

SQL functions are ready-made tools in SQL that help you do calculations, change text, work with dates, or summarize data.

Think of them like shortcuts — instead of writing long code, you just call a function.

Types of Functions

1. Built-in Functions: These are ready-made functions that come with SQL. You don't need to create them just use them directly.

- a. String Functions (e.g., CONCAT, LENGTH, SUBSTRING)
- b. Numeric Functions (e.g., ABS, ROUND, CEIL)
- c. Date and Time Functions (e.g., NOW, DATE_FORMAT, DATEDIFF)
- d. Aggregate Functions (e.g., COUNT, SUM, AVG)

Creating employees table to understand different Commands

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    department VARCHAR(50)  
);
```

```
INSERT INTO employees (emp_id, first_name, last_name, salary, hire_date, department)
```

```
VALUES
```

```
(101, 'Amit', 'Sharma', 45000, '2020-01-15', 'IT'),
```

```
(102, 'Priya', 'Mehta', 52000, '2019-03-10', 'HR'),
```

```
(103, 'Raj', 'Verma', 61000, '2021-07-25', 'Finance'),
```

```
(104, 'Sneha', 'Kapoor', 48000, '2022-11-05', 'IT'),
```

```
(105, 'Arjun', 'Singh', 70000, '2018-06-30', 'Finance');
```

```
select * from employees;
```

String Functions (work with text)

-- 1.Upper : Converts names to uppercase.

```
SELECT *, UPPER(first_name)
FROM employees;
```

-- 2.Lower : Converts names to lowercase.

```
SELECT *, LOWER(last_name)
FROM employees;
```

-- 3.Length : Shows length of first name.

```
SELECT *, LENGTH(first_name)
FROM employees;
```

-- 4. CONCAT : Joins two strings.

```
SELECT *, CONCAT(first_name," ",last_name) as full_name
FROM employees;
```

Numeric Functions (work with numbers)

-- 1. Round : Rounds monthly salary

```
SELECT *, ROUND(salary/12, 2) AS monthly_salary  
  
FROM employees;
```

-- 2. ABS : Shows absolute value

```
SELECT *, ABS(salary - 50000) AS diff  
  
FROM employees;
```

-- 3. Ceil : Rounds a number up to the nearest integer.

```
SELECT CEIL(4.2) AS ceil_example1;
```

```
SELECT CEIL(7.9) AS ceil_example2;
```

-- 4. Floor : Rounds a number down to the nearest integer.

```
SELECT FLOOR(4.2) AS floor_example1;
```

```
SELECT FLOOR(7.9) AS floor_example2;
```

Date/Time Functions (work with dates)

-- 1. **NOW()** → current date and time

```
SELECT NOW() AS current_datetime;
```

-- 2. **CURDATE()** → current date only

```
SELECT CURDATE() AS today_date;
```

-- 3. **YEAR** - extracts year

```
SELECT *, YEAR(hire_date) AS hire_year  
FROM employees;
```

-- 4. **MONTH(join_date)** → extracts month

```
SELECT *, MONTH(hire_date) AS hire_month_number  
FROM employees;
```

```
SELECT *, MONTHNAME(hire_date) AS hire_month_name  
FROM employees;
```

Date/Time Functions (work with dates)

-- 5. **DATEDIFF(end_date, start_date) → difference in days**

```
SELECT *, DATEDIFF(CURDATE(), hire_date) AS days_worked  
FROM employees;
```

```
SELECT *, TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS years_worked  
FROM employees;
```

```
SELECT *, TIMESTAMPDIFF(MONTH, hire_date, CURDATE()) AS months_worked  
FROM employees;
```

Aggregate Functions (work on groups of rows)

-- COUNT(*) --> counts rows

```
SELECT COUNT(*) AS total_employees  
  
FROM employees;
```

-- SUM(salary) --> adds all salaries

```
SELECT SUM(salary) AS total_salary  
  
FROM employees;
```

-- AVG(salary) --> average salary

```
SELECT AVG(salary) AS avg_salary  
  
FROM employees;
```

-- MAX(salary) --> highest salary

```
SELECT MAX(salary) AS max_salary  
  
FROM employees;
```

-- MIN(salary) --> lowest salary

```
SELECT MIN(salary) AS min_salary  
  
FROM employees;
```

Video-11

Topics to Cover:

- Group By Clause
- Having Clause
- Difference Between Where and Having Clause
- Group_Concat

Group By

GROUP BY is used to arrange rows into groups based on the values of one or more columns.

- It is usually combined with aggregate functions (COUNT, SUM, AVG, MAX, MIN) to get summary results for each group.

Syntax:

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```


Group By

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

`SELECT country, COUNT(*) AS number
FROM Customers
GROUP BY country;`

country	number
UAE	1
UK	2
USA	2

Creating orders table to understand Group By

```
CREATE TABLE orders (
    cust_id INT,
    order_id INT,
    country VARCHAR(50),
    state VARCHAR(50)
);

INSERT INTO orders (cust_id, order_id, country, state) VALUES
(1, 100, 'USA', 'Seattle'),
(2, 101, 'INDIA', 'Delhi'),
(2, 103, 'INDIA', 'Delhi'),
(4, 108, 'USA', 'WDC'),
(5, 109, 'UK', 'London'),
(4, 110, 'USA', 'WDC'),
(3, 120, 'INDIA', 'Delhi'),
(2, 121, 'INDIA', 'Goa'),
(1, 131, 'USA', 'Seattle'),
(6, 142, 'USA', 'Seattle'),
(7, 150, 'USA', 'Seattle');
```

-- Example-1: Calculate total order placed Country wise.

```
SELECT country, count(order_id) as order_count  
  
FROM orders  
  
GROUP BY country;
```

-- You can also use * in place of column name.

```
SELECT country, count(*) as order_count  
  
FROM orders  
  
GROUP BY country;
```

-- Example-2: Calculate total order placed State wise.

```
SELECT state, count(order_id) as order_count  
  
FROM orders  
  
GROUP BY state;
```

-- Example-3: Calculate total order placed by both Country & State.

```
SELECT country, state, count(order_id) as order_count  
  
FROM orders  
  
GROUP BY country, state;
```

Creating employees table to understand Group By

```
CREATE TABLE employees (  
  
    emp_id INT PRIMARY KEY,  
  
    first_name VARCHAR(50),  
  
    last_name VARCHAR(50),  
  
    department VARCHAR(50),  
  
    salary DECIMAL(10,2),  
  
    hire_date DATE );  
  
  
INSERT INTO employees (emp_id, first_name, last_name, department, salary, hire_date)  
  
VALUES  
  
(1, 'Amit', 'Sharma', 'IT', 60000, '2020-01-15'),  
  
(2, 'Neha', 'Rao', 'IT', 75000, '2021-03-10'),  
  
(3, 'Vikram', 'Joshi', 'IT', 70000, '2019-07-25'),  
  
(4, 'Anita', 'Deshmukh', 'HR', 80000, '2022-05-05'),  
  
(5, 'Rohan', 'Iyer', 'HR', 50000, '2018-11-20'),  
  
(6, 'Karan', 'Patel', 'HR', 55000, '2021-02-18'),  
  
(7, 'Meera', 'Kapoor', 'Sales', 45000, '2020-09-12'),  
  
(8, 'Sanjay', 'Verma', 'Sales', 48000, '2019-11-30'),  
  
(9, 'Priya', 'Nair', 'Marketing', 65000, '2021-07-14'),  
  
(10, 'Arjun', 'Singh', 'Marketing', 62000, '2022-03-08');  
  
  
Select * from employees;
```

-- Example-4: Calculate Total salary per department

```
SELECT department, sum(salary)
FROM employees
GROUP BY department;
```

-- Example-5: Calculate Average salary per department

```
SELECT department, avg(salary)
FROM employees
GROUP BY department;
```

-- Example-6: Group by with multiple aggregation

```
SELECT department,
       COUNT(*) AS total_employees,
       AVG(salary) AS avg_salary,
       MIN(salary) AS lowest_salary,
       MAX(salary) AS highest_salary
FROM employees
GROUP BY department;
```

Having

The HAVING clause in SQL is used to filter grouped records.

The HAVING clause in SQL is used to filter groups of rows after applying GROUP BY and is especially useful when applying conditions to aggregate functions like SUM(), COUNT(), AVG(), etc.

Where Clause V/s Having Clause

- WHERE filters rows before grouping.
- HAVING filters groups after grouping.
- HAVING Typically used with GROUP BY, but can also be used without it (then it acts like WHERE).

Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
```

By Amit Kumar

Understanding Difference Between Where and Having

```
Select * from employees;
```

```
Select *  
from employees  
Where department <> 'IT';
```

```
Select department, sum(salary)  
from employees  
group by department;
```

```
Select department, sum(salary)  
from employees  
group by department  
having sum(salary) > 100000;
```

GROUP_CONCAT

- GROUP_CONCAT() function returns a string with a concatenated non-NULL value from a group.
- It is used to concatenate values from multiple rows within a specific column into a single string.
- It is particularly useful for combining and displaying related data in a compact format.

Syntax:

```
GROUP_CONCAT([DISTINCT] expression  
             [ORDER BY expression ASC|DESC]  
             [SEPARATOR string])
```


Group_Concat Examples:

– Concat state for each country

```
SELECT country, GROUP_CONCAT(state) AS state_in_country
```

```
FROM orders
```

```
GROUP BY country;
```

– Concat distinct state for each country

```
SELECT country, GROUP_CONCAT(DISTINCT state) AS state_in_country
```

```
FROM orders
```

```
GROUP BY country;
```

– Concat distinct state for each country and sort it in descending order

```
SELECT country, GROUP_CONCAT(DISTINCT state ORDER BY STATE DESC) AS state_in_country
```

```
FROM orders
```

```
GROUP BY country;
```

– Concat distinct state for each country and sort it in ascending order and use "|" as separator

```
SELECT country, GROUP_CONCAT(DISTINCT state ORDER BY STATE SEPARATOR "|") AS
```

```
state_in_country
```

```
FROM orders
```

```
GROUP BY country;
```

Video-12

Topics to Cover:

- Subquery in SQL

Subquery

A subquery in SQL is a query nested inside another query. It is often called an inner query, while the main query is the outer query.

Subqueries allow you to use the result of one query as input to another, making your SQL more powerful and flexible.

Syntax:

```
SELECT column_list
```

```
FROM table
```

```
WHERE expression operator (SELECT column_list FROM table WHERE condition);
```

- **Outer query** → the main query that uses the result of the subquery.
- **Inner query (subquery)** → the query inside parentheses, executed first.
- **Operator** → defines how the outer query compares with the subquery result (**IN**, **NOT IN**, **ANY**, **ALL**, **EXISTS**, etc.).

Creating employees table to understand different Commands

```
CREATE TABLE employees (  
    emp_id    INT PRIMARY KEY,  
    name      VARCHAR(50),  
    department VARCHAR(50),  
    salary    INT  
);
```

-- Sample data

```
INSERT INTO employees VALUES  
(1, 'Amit', 'HR', 40000),  
(2, 'Priya', 'IT', 60000),  
(3, 'John', 'Finance', 55000),  
(4, 'Emma', 'IT', 70000),  
(5, 'Rahul', 'HR', 30000),  
(6, 'Neha', 'Finance', 65000);
```

```
SELECT * FROM employees;
```

-- Find employees who are getting more salary then Priya:

-- Step-1

```
SELECT salary  
FROM employees  
WHERE name = "Priya";
```

-- Step-2

```
SELECT *  
FROM employees  
WHERE salary > 60000;
```

-- Combine step

```
SELECT *  
FROM employees  
WHERE salary > (SELECT salary FROM employees WHERE name = "Priya");
```

IN: The IN operator allows you to specify multiple values in a WHERE clause. It returns true if a value matches any value in a list.

-- Find employees who are in HR or IT department

```
SELECT *  
FROM employees  
WHERE department IN ("HR" , "IT");
```

NOT IN: The NOT IN operator excludes the values in the list. It returns true if a value does not match any value in the list.

-- Find employees who are not in HR or IT department

```
SELECT *  
FROM employees  
WHERE department NOT IN ("HR" , "IT");
```

-- One more example of IN : Find employees in the same department as Priya:

```
SELECT *  
FROM employees  
WHERE department IN (SELECT department FROM employees WHERE name = 'Priya');
```

-- One more example of NOT IN : Find employees not in the same department as Priya:

```
SELECT *  
FROM employees  
WHERE department NOT IN (SELECT department FROM employees WHERE name = 'Priya');
```

ANY: The ANY operator returns true if any subquery value meets the condition.

-- Find employees earning more than any HR employee:

```
SELECT *  
FROM employees  
WHERE salary > ANY (SELECT salary FROM employees WHERE department = 'HR');
```

-- note: Equivalent to "greater than the minimum HR salary."

ALL: The ALL operator returns true if all subquery value meets the condition.

-- Find employees earning more than all HR employees:

```
SELECT * FROM employees
```

```
WHERE salary > ALL (SELECT salary FROM employees WHERE department = 'HR');
```

EXISTS: The EXISTS operator returns true if the subquery returns one or more records.

-- Find employees who belong to a department that has at least one employee earning above 60,000:

```
SELECT DISTINCT e.department
```

```
FROM employees e
```

```
WHERE EXISTS (SELECT 1 FROM employees WHERE department = e.department AND salary > 60000);
```


NOT EXISTS: The NOT EXISTS operator returns true if the subquery returns no records.

-- Find departments where no employee earns above 60,000:

```
SELECT DISTINCT e.department
```

```
FROM employees e
```

```
WHERE NOT EXISTS (SELECT 1 FROM employees WHERE department = e.department AND salary > 60000);
```

Video-13

Topics to Cover:

- Case Statement in SQL

Case Statement

- It's like an IF-THEN-ELSE inside SQL.
- You use it when you want different outputs depending on a condition.
- It checks conditions one by one, and stops at the first match.
- If nothing matches, it gives the ELSE value (or NULL if no ELSE is written).

The **CASE** statement in MySQL comes in two main forms:

1. Simple CASE

- Compares one expression against multiple possible values.
- Works like a switch statement in programming.

2. Searched CASE

```
CASE expression
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ...
  ELSE resultN
END
```

ns).
rators

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ...
  ELSE resultN
END
```

Creating student table to understand Simple Case:

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    department VARCHAR(10) );
```

```
INSERT INTO employees (department)  
VALUES  
('HR'), ('IT'), ('FIN'), ('SALES'), ('MKT'), ('IT'), ('HR');
```

```
Select * from employees;
```

```
SELECT emp_id, department,  
    CASE department  
        WHEN 'HR' THEN 'Human Resources'  
        WHEN 'IT' THEN 'Information Technology'  
        WHEN 'FIN' THEN 'Finance'  
        ELSE 'Other'
```

```
END AS department_full
```

Creating student table to understand Searched Case:

```
CREATE TABLE students (  
stu_id INT,  
stu_name VARCHAR(20),  
marks FLOAT  
);
```

```
INSERT INTO students  
VALUES (1,"ankit",86), (2,"rahul",58),(3,"amit",95),(4,"rajesh",80),(5,"nitin",65);
```

```
SELECT * FROM students;
```

/* Grading:

marks > 90 : grade A+

marks >= 80 and marks <90 so grade will A

marks >= 70 and marks <80 so grade will B+

marks >= 60 and marks <70 so grade will B

marks < 60 so grade will be C */

Creating student table to understand Searched Case:

```
SELECT *,  
    CASE  
        WHEN marks>90 THEN "A+"  
        WHEN marks>=80 AND marks<90 THEN "A"  
        WHEN marks>=70 AND marks <80 THEN "B+"  
        WHEN marks >=60 AND marks <70 THEN "B"  
        ELSE "C"  
    END AS grade  
FROM students;
```

Video-14

Topics to Cover:

- Joins in SQL

Joins

customer			
<i>customer_id</i>	<i>first_name</i>	<i>last_name</i>	<i>email_address</i>
1	Bob	Bobbinson	bob@email.com
2	Mark	Markson	mark@email.com
3	Jack	Jackson	jack@email.com
4	Jane	Janeson	jane@email.com

NC

sale			
<i>sale_id</i>	<i>customer_id</i>	<i>date_of_sale</i>	<i>total_amount</i>
1	1	01.01.2022	50
2	1	05.01.2022	75
3	2	07.01.2022	20
4	3	01.02.2022	30
5	3	03.02.2022	90

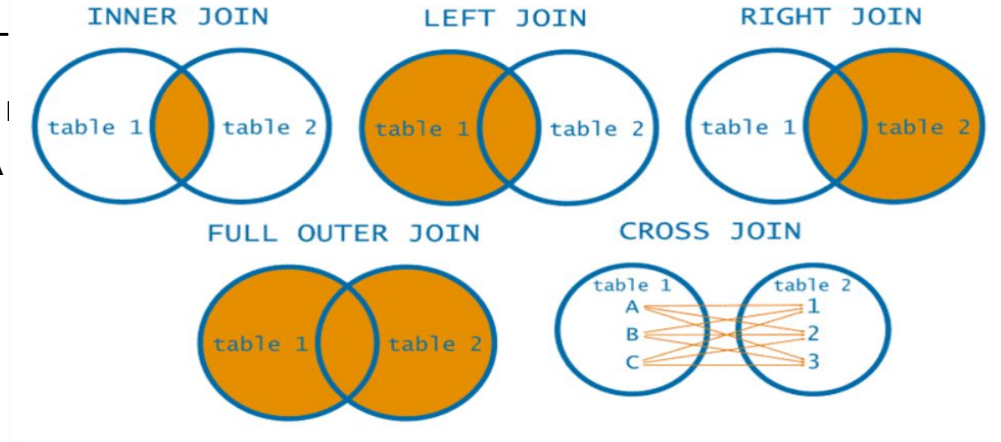
Output Table

<i>first_name</i>	<i>last_name</i>	<i>date_of_sale</i>	<i>total_amount</i>
Bob	Bobbinson	01.01.2022	50
Bob	Bobbinson	05.01.2022	75
Mark	Markson	07.01.2022	20
Jack	Jackson	01.02.2022	30
Jack	Jackson	03.02.2022	90

Main Types of Joins in MySQL

1. INNER JOIN - Returns rows that have matching values in both tables.
2. LEFT JOIN - Returns all rows from the left table, and matching rows from the right table.
3. RIGHT JOIN - Returns all rows from the right table, and matching rows from the left table.
4. FULL JOIN - Returns all rows when there is a match in either left or right table.
5. CROSS JOIN - Returns every row from table A combined with every row from table B.

6. SELF JOIN - A join where a table is joined to itself.



| table A combined with

Creating Department & Employees table to understand Join

-- Departments table

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

-- Employees table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

```
INSERT INTO departments
VALUES
    (1,'Human Resources'),
    (2,'Information Technology'),
    (3,'Finance'),
    (4,'Sales');
```

```
INSERT INTO employees
VALUES
    (101,'Amit', 1), -- HR
    (102,'Priya', 2), -- IT
    (103,'Ravi', 3), -- Finance
    (104,'Sneha', NULL), -- No department
    (105,'Arjun', 2), -- IT
    (106,'Meera', 4); -- Sales
```

1. INNER JOIN: Returns rows where there is a match in both tables. Excludes non-matching rows.

Syntax:

```
SELECT column_list  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```

```
SELECT employees.emp_id, employees.name, departments.dept_name  
FROM employees  
INNER JOIN departments  
ON employees.dept_id = departments.dept_id;
```

-- With Alias

```
SELECT e.emp_id, e.name, d.dept_name  
FROM employees e  
INNER JOIN departments d  
ON e.dept_id = d.dept_id;
```

2. LEFT JOIN: Returns all rows from the left table, and matched rows from the right table.

- Non-matching rows from the right table show as NULL.

Syntax:

```
SELECT column_list  
FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column;
```

```
SELECT e.emp_id, e.name, d.dept_name  
FROM employees e  
LEFT JOIN departments d  
ON e.dept_id = d.dept_id;
```

3. RIGHT JOIN: Returns all rows from the right table, and matched rows from the left table.

- Non-matching rows from the left table show as NULL.

Syntax:

```
SELECT column_list  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

```
SELECT e.emp_id, e.name, d.dept_name  
FROM employees e  
RIGHT JOIN departments d  
ON e.dept_id = d.dept_id;
```

4. FULL JOIN: MySQL doesn't support FULL OUTER JOIN directly.

- We can emulate it using UNION of LEFT and RIGHT joins:

Syntax:

LEFT JOIN

UNION

RIGHT JOIN

```
SELECT e.emp_id, e.name, d.dept_name
```

```
FROM employees e
```

```
LEFT JOIN departments d
```

```
ON e.dept_id = d.dept_id;
```

```
UNION
```

```
SELECT e.emp_id, e.name, d.dept_name
```

```
FROM employees e
```

```
RIGHT JOIN departments d
```


Video-15

Topics to Cover:

- Joins in SQL
 - 5. CROSS JOIN
 - 6. SELF JOIN

Topics to cover:

- Joins in SQL
 - 5. CROSS JOIN
 - 6. SELF JOIN

5. CROSS JOIN: Produces the Cartesian product of both tables.
Every row from the first table combines with every row from the second.

Syntax:

```
SELECT column_list  
FROM table1  
CROSS JOIN table2;
```

Note: If 3 employees and 4 departments exist, result = 12 rows



Example Tables

1. Students Table

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50)  
);  
  
INSERT INTO Students (student_id, student_name) VALUES  
(1, 'Alice'),  
(2, 'Bob');
```

2. Courses Table

```
CREATE TABLE Courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50)  
);  
  
INSERT INTO Courses (course_id, course_name) VALUES  
(101, 'Math')
```

CROSS JOIN Example

```
SELECT student_id, student_name, course_name  
FROM Students  
CROSS JOIN Courses;
```

6. SELF JOIN: A self join is simply a regular join where a table is joined to itself. You must use aliases to differentiate the two instances of the same table.

Syntax:

```
SELECT a.column_list, b.column_list  
FROM table_name a  
JOIN table_name b  
ON a.common_column = b.common_column;
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50) NOT NULL,  
    manager_id INT,  
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id)  
);
```

```
INSERT INTO employees (name, manager_id)  
VALUES
```

```
('Amit', NULL), -- Amit is the top-level manager (no manager)  
( 'Priya', 1),  -- Priya reports to Amit  
( 'Ravi', 1),   -- Ravi reports to Amit  
( 'Sneha', 2),  -- Sneha reports to Priya  
( 'Arjun', 3);  -- Arjun reports to Ravi
```

```
SELECT * FROM employees;
```

```
SELECT e1.emp_id, e1.name AS Employee, e2.name AS Manager  
FROM employees e1  
LEFT JOIN employees e2  
ON e1.manager_id = e2.emp_id;
```


Video-16

Topics to Cover:

- Window Functions in SQL

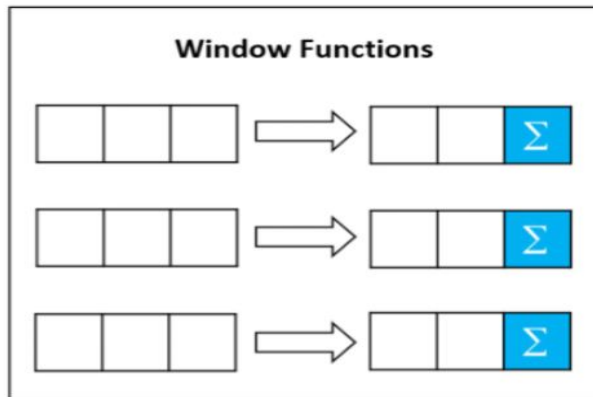
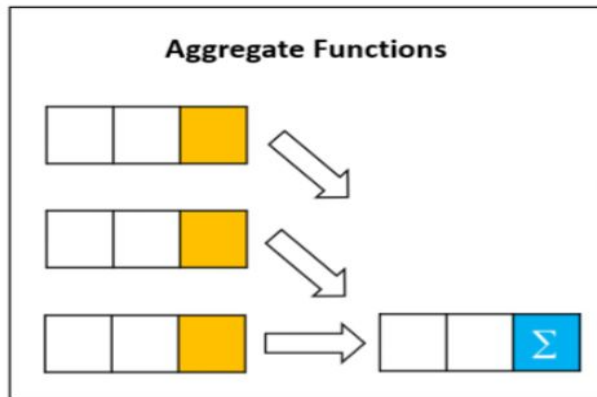
Window Functions

Why we need Window Functions?

When we use group by instead of window function we are getting aggregated data, here actually we are losing some of our attribute or data. That was the biggest disadvantage.

In order to give more flexibility plus keeping the originality of the data we are using the window function.

- **Definition**
clause.



the OVER()

Syntax:

```
SELECT column_name(s),  
function_name (expression) OVER ( [PARTITION BY column]  
                                   [ORDER BY column]  
                                   [ROWS or RANGE frame_spec] )  
FROM table_name;
```

- **function_name** : Select a function : Aggregation Functions, Ranking Functions, Value Functions.
- **Expression** : is the name of the column that we want the window function operated on. This may not be necessary depending on what window function is used.
- **OVER** : Define a Window.

PARTITION BY : Divides rows into groups (like GROUP BY but for window functions).

ORDER BY : Defines the order of rows within each partition. This is optional.

Different Types of Window Functions

Aggregation Functions:

- SUM()
- COUNT()
- MIN()
- MAX()
- AVG()

Ranking Functions:

- ROW_NUMBER(): Assigns a unique row number to each row, ranking start from 1 and keep increasing till the end of last row
- RANK(): Assigns a rank to each row. Rows with equal values receive the same rank, with the next row receiving a rank which skips the duplicate rankings.
- DENSE_RANK(): Similar to RANK(), but does not skip rankings if there are duplicates.

Value Functions:

- LEAD(): Returns the value of the next row.
- LAG(): Returns the value of the previous row
- FIRST_VALUE(): Returns the first value in the window
- LAST_VALUE(): Returns the last value in the window

Create sample table to understand Aggregate Window Functions

```
CREATE TABLE sales (  
    id INT,  
    employee VARCHAR(50),  
    department VARCHAR(50),  
    sales_amount INT  
);
```

```
INSERT INTO sales VALUES  
(1, 'Alice', 'Electronics', 500),  
(2, 'Bob', 'Electronics', 800),  
(3, 'Charlie', 'Electronics', 800),  
(4, 'David', 'Grocery', 400),  
(5, 'Eva', 'Grocery', 300),  
(6, 'Frank', 'Clothing', 300),  
(7, 'Grace', 'Clothing', 700);
```

Aggregate Functions

```
SELECT employee, department, sales_amount,  
       SUM(sales_amount) OVER (PARTITION BY department ORDER BY department) AS dept_sum,  
       AVG(sales_amount) OVER (PARTITION BY department ORDER BY department) AS dept_avg,  
       COUNT(*)          OVER (PARTITION BY department ORDER BY department) AS dept_count,  
       MIN(sales_amount) OVER (PARTITION BY department ORDER BY department) AS dept_min,  
       MAX(sales_amount) OVER (PARTITION BY department ORDER BY department) AS dept_max  
FROM sales;
```

/* Note: You can use a single window function or combine multiple window functions in your query, depending on your specific requirement. */

-- single Aggregate Functions

```
SELECT employee, department, sales_amount,  
       SUM(sales_amount) OVER (PARTITION BY department ORDER BY department) AS dept_sum,
```

Create sample table to understand Ranking Window Functions

```
CREATE TABLE shop_sales (  
    sales_date DATE,  
    shop_id VARCHAR(5),  
    sales_amount DECIMAL(10,2)  
);
```

```
INSERT INTO shop_sales VALUES  
(  
'2024-01-01', 'S001', 400.00),  
(  
'2024-01-02', 'S001', 400.00),  
(  
'2024-01-03', 'S001', 300.00),  
(  
'2024-01-04', 'S001', 200.00),  
(  
'2024-01-05', 'S002', 600.00),  
(  
'2024-01-06', 'S002', 600.00),  
(  
'2024-01-07', 'S003', 500.00),  
(  
'2024-01-07', 'S003', 500.00),  
(  
'2024-01-07', 'S003', 300.00);
```

Ranking Functions:

-- Without partition by

```
SELECT *,  
    ROW_NUMBER() OVER(ORDER BY sales_amount) AS row_num,  
    RANK() OVER(ORDER BY sales_amount) AS rank_val,  
    DENSE_RANK() OVER(ORDER BY sales_amount) AS dense_rank_val  
FROM shop_sales;
```

-- With partition by

```
SELECT *,  
    ROW_NUMBER() OVER(PARTITION BY shop_id ORDER BY sales_amount DESC) AS row_num,  
    RANK() OVER(PARTITION BY shop_id ORDER BY sales_amount DESC) AS rank_val,  
    DENSE_RANK() OVER(PARTITION BY shop_id ORDER BY sales_amount DESC) AS dense_rank_val  
FROM shop_sales;
```


/* Note: You can use a single window function or combine multiple window functions in your query, depending on your specific requirement. */

-- Row_Number()

```
SELECT *,  
    ROW_NUMBER() OVER(PARTITION BY shop_id ORDER BY sales_amount DESC) AS row_num  
FROM shop_sales;
```

-- Rank()

```
SELECT *,  
    RANK() OVER(PARTITION BY shop_id ORDER BY sales_amount DESC) AS rank_val  
FROM shop_sales;
```

-- Dense_Rank()

```
SELECT *,  
    DENSE_RANK() OVER(PARTITION BY shop_id ORDER BY sales_amount DESC) AS dense_rank_val  
FROM shop_sales;
```

Value Functions:

-- Without partition by

```
SELECT employee, department, sales_amount,  
       LAG(sales_amount) OVER (ORDER BY sales_amount) AS prev_sales,  
       LEAD(sales_amount) OVER (ORDER BY sales_amount) AS next_sales,  
       FIRST_VALUE(sales_amount) OVER (ORDER BY sales_amount) AS first_sales,  
       LAST_VALUE(sales_amount) OVER (ORDER BY sales_amount  
       ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_sales  
FROM sales;
```

-- With partition by

```
SELECT employee, department, sales_amount,  
       LAG(sales_amount, 1) OVER (PARTITION BY department ORDER BY sales_amount) AS prev_sales,  
       LEAD(sales_amount, 1) OVER (PARTITION BY department ORDER BY sales_amount) AS next_sales,  
       FIRST_VALUE(sales_amount) OVER (PARTITION BY department ORDER BY sales_amount) AS first_sales,  
       LAST_VALUE(sales_amount) OVER (PARTITION BY department ORDER BY sales_amount
```

Frame Clause

/* - The frame start can be:

- UNBOUNDED PRECEDING (starts at the first row of the partition)
- N PRECEDING (starts N rows before the current row)
- CURRENT ROW (starts at the current row)

- The frame end can be:

- UNBOUNDED FOLLOWING (ends at the last row of the partition)
- N FOLLOWING (ends N rows after the current row)
- CURRENT ROW (ends at the current row) */

Video-17

Topics to Cover:

- UNION Vs UNION ALL in SQL

UNION

The SQL UNION clause/operator is used to combine/concatenate the results of two or more SELECT statements without returning any duplicate rows and keeps unique records.

To use this UNION clause each SELECT statement must have :

- The same number of columns selected and expressions
- The same data type and
- Have them in the same order

Syntax

```
SELECT column1, column2, ... FROM table1
```

```
UNION
```

```
SELECT column1, column2, ...FROM table2;
```

UNION ALL

In UNION ALL everything is same as UNION , it combines/concatenate two or more table but keeps all records, including duplicates.

- It's generally faster because MySQL doesn't need to check for uniqueness.

Syntax

```
SELECT column1, column2, ... FROM table1  
UNION ALL  
SELECT column1, column2, ...FROM table2;
```

Create sample tables to understand Union and Union All

```
CREATE TABLE employees (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50) );
```

```
CREATE TABLE managers (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50) );
```

```
INSERT INTO employees (name)  
VALUES ('Alice'), ('Bob'), ('Charlie'), ('David');
```

```
INSERT INTO managers (name)  
VALUES ('Bob'), ('Charlie'), ('Eve'), ('Frank');
```

```
SELECT name FROM employees;
```

```
SELECT name FROM managers;
```

UNION:

```
SELECT name FROM employees  
UNION  
SELECT name FROM managers;
```

UNION ALL:

```
SELECT name FROM employees  
UNION ALL  
SELECT name FROM managers;
```


Video-18

Topics to Cover:

- Common Table Expression(CTE) in SQL

Common Table Expression(CTE)

- A CTE is like a temporary table that you can reference within a query using the **WITH** clause.
- CTEs are not stored permanently; they only exist during query execution.
- It can be used in SELECT, INSERT, UPDATE, or DELETE statements.
- It makes complex queries easier to read, maintain, and reuse.
- The WITH clause can include one or more CTEs separated by commas.
- MySQL introduced CTEs starting from version 8.0.

Syntax:

```
WITH cte_name AS (  
    SELECT column1, column2  
    FROM table_name  
)  
SELECT * FROM cte_name;
```

A CTE is defined using the WITH keyword, followed by the CTE name and a query. The CTE can then be referred to by its name elsewhere in the query.

Create sample table to understand CTE

```
CREATE TABLE employees (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10,2)  
);  
  
-- Insert sample data  
INSERT INTO employees (name, department, salary) VALUES  
('Alice', 'HR', 50000),  
('Bob', 'IT', 60000),  
('Charlie', 'Finance', 55000),  
('David', 'IT', 70000),  
('Eva', 'HR', 48000);  
  
select * from employees;
```

-- Example-1 : Let's find employees earning above the average salary.

```
WITH avg_salary AS (  
    SELECT AVG(salary) AS avg_sal  
    FROM employees  
)  
SELECT e.name, e.department, e.salary  
FROM employees e, avg_salary a  
WHERE e.salary > a.avg_sal;
```

-- Example-2 : Find Department wise salary and department wise max salary

/* another example with multiple CTE for first we have to use WITH clause
for 2nd we do not need to use WITH clause as we have used comma for separation */

```
with dept_wise_salary as ( select department, sum(salary) as total_salary  
    from employees group by department),  
    dept_wise_max_salary as ( select department , max(salary) as max_salary  
    from employees group by department)
```

Video-19

Topics to Cover:

- Recursive Common Table Expression(CTE) in SQL

Recursive CTE

This is a CTE that references itself. In other words, the CTE query definition refers back to the CTE name, creating a loop that ends when a certain condition is met.

A recursive CTE has three elements:

- **Non-recursive term:** It's a CTE query definition that forms the base result set of the CTE structure.
- **Recursive term:** One or more CTE query definitions joined with non-recursive term using UNION or UNION ALL operator.
- **Termination check:** the recursion stops when no rows are returned from the previous iteration.

Syntax

```
WITH RECURSIVE cte_name AS (  
    CTE_query_definition -- non recursive term(base query/anchor member)  
    UNION ALL  
    recursive_query_definition -- recursive term (recursive query / recursive member)  
)  
SELECT * FROM cte_name;
```

Recursive CTE

Example - 1

```
WITH RECURSIVE cte_count AS (  
    SELECT 1 AS n -- non recursive term(base query/anchor member)  
    UNION ALL  
    SELECT n+1 FROM cte_count -- recursive term (recursive query / recursive member)  
    Where n<5 ) -- Termination check  
  
SELECT * FROM cte_count;
```

Recursive CTE

Example - 2 : Recursive Date Generator

```
CREATE TABLE CalendarDates (  
    DateValue DATE PRIMARY KEY  
);  
  
WITH RECURSIVE DatesCTE AS (  
    SELECT DATE('2026-01-01') AS DateValue -- Anchor: start with the first day of the month  
  
    UNION ALL  
  
    SELECT DateValue + INTERVAL 1 DAY -- Recursive: add one day until the end of the month  
    FROM DatesCTE  
    WHERE DateValue < '2026-01-05'  
)  
SELECT * FROM DatesCTE;
```


Recursive CTE

Example - 3 Finding Employees Hierarchy

```
CREATE TABLE employees (  
    emp_id int PRIMARY KEY,  
    emp_name VARCHAR(50) NOT NULL,  
    manager_id INT  
);
```

```
INSERT INTO employees (emp_id, emp_name, manager_id) VALUES  
(1, 'Alice', NULL),  
(2, 'Bob', 1),  
(3, 'Carol', 2),  
(4, 'David', 6),  
(5, 'Eva', 4),  
(6, 'Tom', 1),  
(7, 'Frank', 5);
```

Recursive CTE

```
WITH RECURSIVE EmployeeHierarchy AS (  
    -- Anchor: start with root manager(s)  
    SELECT emp_id, emp_name, manager_id  
    FROM employees  
    WHERE emp_id = 7  
  
    UNION ALL  
  
    -- Recursive: find direct reports  
    SELECT employees.emp_id, employees.emp_name, employees.manager_id  
    FROM employees  
    JOIN EmployeeHierarchy  
        ON employees.emp_id = EmployeeHierarchy.manager_id  
)  
SELECT * FROM EmployeeHierarchy;
```

Video-20

Topics to Cover:

- Views(Virtual Table) in SQL

Views

- A view in SQL is a virtual table based on the result-set of an SQL statement.
- It does not store data physically.
- It contains rows and columns, just like a real table.
- The fields in a view are fields from one or more real tables in the Database.

Here are some key points about views:

- You can add SQL functions, WHERE, and JOIN statements to a view and display the data as if the data were coming from one single table.
- A view always shows up-to-date data. The database engine recreates the data every time a user queries a view.
- Views can be used to encapsulate complex queries, presenting users with a simpler interface to the data.
- They can be used to restrict access to sensitive data in the underlying tables, presenting only non-sensitive data to users.

Syntax

```
CREATE VIEW view_name AS  
SELECT column1, column2
```

Create sample table to understand Views

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees (emp_id, emp_name, department, salary)  
VALUES  
(1, 'amit', 'it', 60000),  
(2, 'priya', 'hr', 45000),  
(3, 'ravi', 'finance', 55000),  
(4, 'sneha', 'it', 65000),  
(5, 'neha', 'hr', 75000),  
(6, 'ankita', 'it', 48000);
```

Create sample table to understand Views

Example-1 : Create view for IT department

```
CREATE VIEW it_employees AS  
SELECT * FROM employees  
WHERE department = 'it';  
  
SELECT * FROM it_employees;
```

Example-2 : Create view department wise salary

```
CREATE VIEW dept_employees AS  
SELECT department, SUM(salary)  
FROM employees  
GROUP BY department;  
  
SELECT * FROM dept_employees;
```

DROP A View

```
DROP VIEW view_name;
```

Video-21

Topics to Cover:

- Indexes in SQL

Indexes

In MySQL, indexes make queries faster by allowing the database to quickly locate rows without scanning the entire table.

- They are like a book's index—helping you jump directly to the needed information.
- Without indexes, MySQL must perform a full table scan, which is slow for large datasets
- It is a powerful way to optimize the performance of a database.
- It improves the speed of data retrieval operations on a database table.
- Indexes are used to quickly locate data without having to search every row in a table each time a database table is accessed.

Syntax to create index:

```
CREATE INDEX index_name  
ON table_name(column_name);
```


Indexes

```
SELECT COUNT(*) FROM swiggy;
```

```
SELECT * FROM Swiggy LIMIT 5;
```

Example-1

```
SELECT COUNT(*) FROM swiggy  
WHERE item = "Roasted Papad";
```

-- Create Index

```
CREATE INDEX item_index  
ON swiggy(item);
```

See Index

```
SHOW INDEX FROM swiggy;
```

Indexes

Example-2

```
SELECT COUNT(*) FROM swiggy  
WHERE cuisine = "Biryani,Chinese";
```

-- Create Index

```
CREATE INDEX cuisine_index  
ON swiggy(cuisine);
```

Drop Index

```
ALTER TABLE swiggy  
DROP INDEX item_index;
```

Video-22

Topics to Cover:

- Pivot and Unpivot in SQL

Pivot and Unpivot

Pivoting and unpivoting are techniques used to transform data from a row-wise format to a column-wise format (pivot) or vice versa (unpivot). These operations are often used when you want to reshape your data for reporting or analysis

Pivot
you

Country	Year	Profit (USD)					
USA	2020	495875			Country	2020	2021
USA	2021	459875			USA	495875	459875
France	2020	145685			France	145685	201457
France	2021	201457			Germany	178563	165478
Germany	2020	178563					
Germany	2021	165478					

Useful when

Pivot and Unpivot

Un
wh
and

Country	2020	2021
USA	495875	459875
France	145685	201457
Germany	178563	165478



Country	Year	Profit (USD)
USA	2020	495875
USA	2021	459875
France	2020	145685
France	2021	201457
Germany	2020	178563
Germany	2021	165478

This is useful
further

Pivoting:

```
CREATE TABLE sales (  
    product VARCHAR(10),  
    month VARCHAR(10),  
    sales INT );
```

```
INSERT INTO sales (product, month, sales) VALUES  
( 'A', 'Jan', 100),  
( 'A', 'Feb', 120),  
( 'B', 'Jan', 90),  
( 'B', 'Feb', 110);
```

```
SELECT * FROM sales;
```

```
SELECT product,  
    SUM(CASE WHEN month = 'Jan' THEN sales END) AS Jan_sales,  
    SUM(CASE WHEN month = 'Feb' THEN sales END) AS Feb_sales
```

UnPivoting

```
CREATE TABLE student_scores (  
    student_id INT,  
    math_score INT,  
    science_score INT,  
    history_score INT );
```

```
INSERT INTO student_scores (student_id, math_score, science_score, history_score) VALUES  
    (1, 95, 88, 75),  
    (2, 88, 92, 80),  
    (3, 90, 85, 78);
```

```
SELECT * FROM student_scores;
```

```
SELECT student_id, 'math' AS subject, math_score AS score FROM student_scores  
UNION ALL
```

```
SELECT student_id, 'science' AS subject, science_score AS score FROM student_scores
```

👏 **Congrats on completing the Complete SQL Course! You've leveled up your data skills—now go use them to build, analyze, and create amazing things. Keep learning, keep coding, and keep shining!**



Next Step:

- 1. Subscribe My youtube channel: [Click Here](#)**
- 2. Create SQL Projects : [Click Here](#)**
- 3. Practice SQL Interview Questions : [Click Here](#)**

By Amit Kumar

By Amit Kumar

जय बजरंग बली

AMIT KUMAR

 Want to connect with me? Check out these links: 

 Youtube: <https://www.youtube.com/@OfficialAccountAmitKumar>

 LinkedIn: <https://www.linkedin.com/in/amit1990>